

Exemple, LecteursEtEcrivains (v4)

Fabrice.Kordon@lip6.fr



Objectif de la variante

2

Supprimer des attentes semi-actives

- Celles des écrivains
- L'écriture devient bloquante
- La lecture reste liée à une attente semi-active
- Suppression d'une des deux exceptions
- Première tentative sur la base de l'utilisation des verrous seulement

Modification du tableau noir (ecrireUnMessage)

3

```
public void ecrireUnMessage (int msg, int dest) {
    int i;
    boolean recommencer = true;
    rwl.writeLock().lock();
    try {
        while (recommencer) {
            for (i = 0 ; i < maTaille; i++) {
                if (destinataire[i] == -1) {
                    destinataire[i] = dest;
                    message[i] = msg;
                    recommencer = false;
                    break;
                }
            }
            if (i == maTaille) {
                // le tableau est plein, je dois attendre
                this.trace ("tableau plein, mise en attente du processus");
                rwl.writeLock().unlock();
                synchronized (rwl.writeLock()) {
                    rwl.writeLock().wait(); // mise en attente sur écriture
                }
                rwl.writeLock().lock();
            }
        }
    }
    catch (InterruptedException ie) {
        System.err.println("Ne doit jamais arriver (1)");
    }
    finally {
        rwl.writeLock().unlock();
    }
}
```


Modification du tableau noir (lireUnMessage)

4

```
public int lireUnMessage (int moi) throws RienPourMoiException {
    int valret = 0; // inutile, mais sinon java râle
    int i;
    rwl.readLock().lock();
    try {
        for (i = 0 ; i < maTaille; i++) {
            if (destinataire[i] == moi) {
                valret = message[i];
                break; // sortir de la boucle
            }
        }
        if (i < maTaille) {
            destinataire[i] = -1;
            // Tenter un réveil d'un éventuel écrivain bloqué
            synchronized (rwl.writeLock()) {
                rwl.writeLock().notifyAll();
            }
        } else {
            throw new RienPourMoiException("Rien pour " + moi);
        }
    }
    finally {
        rwl.readLock().unlock();
    }
    return valret;
}
```


Modification du tableau noir (lireUnMessage)

4

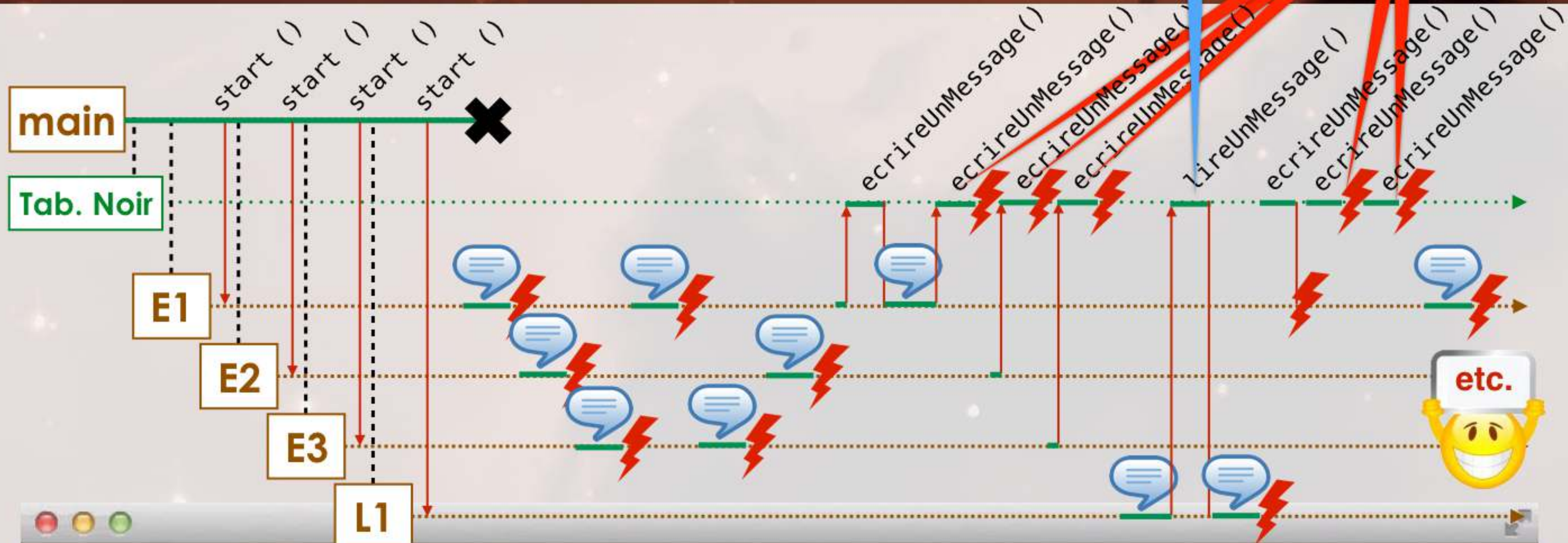
```
public int lireUnMessage (int moi) throws RienPourMoiException {  
    int valret = 0; // inutile, mais sinon java râle  
    int i;  
    rwl.readLock().lock();  
    try {  
        for (i = 0 ; i < maTaille; i++) {  
            if (destinataire[i] == moi) {  
                valret = message[i];  
                break; // sortir de la boucle  
            }  
        }  
        if (i < maTaille) {  
            destinataire[i] = -1;  
            // Tenter un réveil d'un éventuel écrivain bloqué  
            synchronized (rwl.writeLock()) {  
                rwl.writeLock().notifyAll();  
            }  
        } else {  
            throw new RienPourMoiException(moi);  
        }  
    }  
    finally {  
        rwl.readLock().unlock();  
    }  
    return valret;  
}
```

Le reste ne change pas

par rapport à la v3
(mais plus besoin de TropPleinException)

Une exécution détaillée

5



```
$ java LecteursEtEcrivains4 1 1 3
E 1 : initialisé
E 2 : initialisé
E 3 : initialisé
E 1 : envoie 760 à 1 (N° 1)
E 3 : envoie 220 à 1 (N° 1)
E 2 : envoie 239 à 1 (N° 1)
E 1 : envoie 487 à 1 (N° 2)
TableauNoir : tableau plein, mise en attente du processus
TableauNoir : tableau plein, mise en attente du processus
TableauNoir : tableau plein, mise en attente du processus
L 1 : initialisé
L 1 : j'ai reçu 760
TableauNoir : tableau plein, mise en attente du processus
TableauNoir : tableau plein, mise en attente du processus
E 1 : envoie 969 à 1 (N° 3)
```